Hebbian Associative Memory for State Space Models

Jeff Guan
Independent
jeff@jeffguan.com

Abstract

We augment Mamba, a state space model, with a per-layer Hebbian associative memory matrix $W \in \mathbb{R}^{d\times d}$. W accumulates key-value associations via outer-product updates with exponential decay, and injects retrieved content into the residual stream. The mechanism adds 32 lines of code, keeps the training objective, uses $O(d^2)$ memory constant in sequence length, and admits a parallel scan formulation analogous to Mamba's own recurrence, making it nearly drop-in. On PG-19 prose, an 18M-parameter (P) memory model outperforms P-matched baselines (deeper and wider) by 0.05–0.11 nats [TODO: re-evaluate context length scaling claim]. On Python code, the effect is dramatically larger: the memory model achieves a final validation loss 0.94 nats below a P-matched baseline, surpassing the baseline's best-ever performance after only ∼800K training tokens out of 6M. Although trained at 2K tokens, at 16K evaluation windows the gap expands to over 1.0 nats around 8K as W accumulates file-level associations at test time. Code identifiers are key-value pairs bound once and queried repeatedly, exactly what W implements. The benefit scales across sequence length, training compute, and data complexity [TODO: confirm with 100M results]. The linearity of the update rule, with constant memory cost, has potential implications for image and video sequences where long-range entity persistence is similarly associative.

1 Introduction

SSMs (e.g. Mamba [5]) achieve linear-time sequence modeling by compressing history into a fixed-size recurrent state, fundamentally lossy. Transformers avoid this with a full-context KV cache, but at $O(T^2)$ time and $O(T)$ memory.

The question we address is simple: can we give an SSM a persistent associative memory without the quadratic cost of attention?

We answer yes with a simple mechanism. At each Mamba layer, we maintain a matrix $W \in \mathbb{R}^{d\times d}$ that accumulates associations between consecutive hidden states via Hebbian outer-product updates:

$$W_t \leftarrow \gamma W_{t-1} + \text{proj}_{\text{write}}(r_t) \otimes r_{t-1} \tag{1}$$

where r_t is the residual-stream vector at step t and $\gamma = \sigma(\lambda)$ is a learned scalar decay. At each step, the model reads from W and injects the result into the residual stream:

$$r_t \leftarrow r_t + \alpha \cdot \text{proj}_{\text{read}}(W_t \cdot r_t) \tag{2}$$

with α a fixed scalar (0.03 at 18M parameters, 0.05 at 100M). The learned parameters are a scalar λ and two $d \times d$ projection matrices. W itself is recurrent state, not a parameter; initialized to zero at the start of each sequence and accumulating during the forward pass.

This update rule is a linear recurrence in W, admitting an $O(T)$ chunkwise parallel form for training and an $O(d^2)$ per-step recurrent form for inference. Unlike attention, memory cost is constant in sequence length.

Preprint. Under review.

Why Mamba, not Transformers. Transformers have lossless KV-cache memory within the context window — W has no gap to fill. Mamba's lossy recurrent state creates that gap; the mechanisms are complementary: Mamba handles local context efficiently, W handles long-range associations persistently.

Why code. Code is almost entirely associative recall: every identifier is defined once and referenced many times. The key-shift design makes W a natural fit: when the model encounters a usage site, the preceding context matches the predecessor context at the original definition, and W retrieves the Mamba state that followed the definition — a precomputed next-state for an already-seen situation. A Mamba baseline must track all variable bindings through $d_{\text{state}} = 16$ recurrent dimensions; W stores them directly in a $d \times d$ matrix with capacity for $\sim d$ orthogonal associations per layer, growing as $O(\sqrt{N})$ in model parameters N since $d \propto \sqrt{N}$.

Contributions.

- A minimal Hebbian memory mechanism for SSMs: one matrix, one decay, one outer product per step.
- Demonstration that W learns useful associations under standard language modeling training with no resets or auxiliary objectives.
- On prose (PG-19): $+0.05$ nats over a P-matched deep baseline and $+0.11$ nats over wide (2.477 vs. 2.527 vs. 2.589 val loss), gap growing with context length beyond training length. All models are substantially undertrained at 6M tokens; the gap is still widening at the end of training.
- On code (Python) at 18M parameters: a 0.94 nat improvement over a P-matched baseline. The memory model surpasses the baseline's final performance after $\sim$800K training tokens. At 16K tokens (test time), the per-segment gap exceeds 1.0 nats.
- On code at 100M parameters: the memory model leads at every checkpoint over 16.4M training tokens. Best-checkpoint gap: $+0.307$ nats (1.661 vs. 1.968). The 18M memory model (1.848 nats, 6M tokens) outperforms the 100M baseline at *every* point in its training run, including its best-ever checkpoint.
- Evidence that the benefit scales along three axes simultaneously: sequence length, training compute, and model size.

2 Method

Setup. Let $r_t \in \mathbb{R}^d$ denote the residual-stream vector at position t, after the Mamba block at a given layer. Each layer maintains an associative memory matrix $W \in \mathbb{R}^{d \times d}$, initialized to zero at the start of each sequence. W is recurrent state, not a learned parameter.

Write. After each token, W is updated via a Hebbian outer-product rule with exponential decay:

$$W_t \leftarrow \gamma W_{t-1} + \text{proj}_{\text{write}}(r_t) \otimes r_{t-1} \tag{3}$$

The key written at step t is r_{t-1} (the *previous* residual state); the value is $\text{proj}_{\text{write}}(r_t)$ (the *current* residual, projected). This one-step key shift gives W a specific semantic: it stores *predecessor context* $\rightarrow$ *successor state* pairs. When the read query r_t matches a past key r_{s-1}, the retrieved value is $\text{proj}_{\text{write}}(r_s)$ — the Mamba state that immediately followed that predecessor. W is therefore a learned *next-step predictor*: a dynamically accumulated table mapping (context, what-came-after-that-context). $\gamma = \sigma(\lambda) \in (0, 1)$ is a learned scalar decay; associations older than $\sim 1/(1-\gamma)$ steps are exponentially suppressed.

Read. The model retrieves from W by querying with the current state and injecting the result:

$$r_t \leftarrow r_t + \alpha \cdot \text{proj}_{\text{read}}(W_t \cdot r_t) \tag{4}$$

$W_t \cdot r_t$ computes the inner product between r_t and all previously written keys, returning the corresponding weighted sum of values. α is a fixed scalar set per model size.

Parallel form (training). Unrolling the recurrence, reads across a sequence can be expressed as:

$$\text{reads} = (M \odot S)\, V \tag{5}$$

where $S[t, s] = r_t^\top r_{s-1}$ is the similarity matrix, $M[t, s] = \gamma^{t-1-s} \cdot \mathbf{1}[s < t]$ is a causal exponential decay mask, and $V[s] = \text{proj}_{\text{write}}(r_s)$. The entry $S[t, s]$ asks "where in the past did I see a context like mine?" and retrieves what followed. In the *chunkwise* form [11], the sequence is split into chunks of size C. Within each chunk, the intra-chunk contribution uses the quadratic form; across chunks, the running W matrix is passed forward and applied in $O(Cd^2)$ per chunk. Total cost: $O(TC \cdot d + T \cdot d^2)$ time and $O(C^2 + d^2)$ memory — linear in T for fixed C and d. At C=64, d=512, this is $\sim$4$\times$ faster than the quadratic form at T=2048 and $\sim$28$\times$ at T=16384. The recurrence is identical to RetNet/GLA with a scalar decay, so existing hardware-efficient kernels apply directly.

Recurrent form (inference). W is updated one step at a time at $O(d^2)$ per step and $O(d^2)$ memory, constant in sequence length. Training and inference are both linear in T.

Parameters. Each layer adds scalar λ, $\text{proj}_{\text{write}} \in \mathbb{R}^{d \times d}$, and $\text{proj}_{\text{read}} \in \mathbb{R}^{d \times d}$ — roughly $2d^2$ parameters per layer, or $\sim$0.5M at $d = 512$.

Design choices.

- *Fixed α*: A learned injection scale risks instability early in training when W contains noise. α is set by hand (0.03 at 18M, 0.05 at 100M) and not tuned; scaling it modestly with model size reflects that larger W can produce larger-magnitude reads.
- *Separate $\text{proj}_{\text{write}}$ / $\text{proj}_{\text{read}}$*: Allows the model to learn different subspaces for writing and reading, analogous to separate key and value projections in attention. r_t is used directly as the query.
- *No auxiliary objectives*: W trains end-to-end under the standard language modeling loss. The model learns to use W because it is useful for next-token prediction.
- *Per-layer W*: Each layer maintains an independent matrix, allowing different layers to specialize in association type and timescale via their learned γ.
- *Key shift*: Writing at key r_{t-1} rather than r_t is what creates the next-step predictor structure. Standard linear attention binds each position to itself ($k_s = r_s$, as in RetNet and GLA); the shift binds each state to its predecessor, so retrieval surfaces what came *after* the most similar past context rather than the context itself.

3 Experiments and Results

3.1 Synthetic KV Recall

We train a small memory model ($d = 128$, 4 layers) on a synthetic task: a sequence of 32 key-value pairs followed by 32 queries, where the Mamba recurrent state is reset between store and query phases so any successful recall must pass through W.

The model achieves near-perfect recall up to the training distribution (100% at $\leq$16 pairs, 99.8% at 32) and 96% at 64 pairs (2$\times$ training count). Without W, the reset Mamba state has no mechanism to carry associations, giving chance performance ($1/16 \approx 6.2\%$).

3.2 Memory Capacity

We sweep the number of key-value pairs at test time (256 possible keys, 16 values, same trained model):

Naive random-key theory predicts SNR $\approx \sqrt{d/k} \approx 1.4$ at $k = 64$, corresponding to $\sim$50–70% recall. The model achieves 96%, indicating learned projections structure keys to reduce interference. At 256 pairs, 18.8% accuracy corresponds to $\sim$48 correct, or $\sim$12 per layer across 4 independent W matrices — within single-matrix capacity.

Pairs	Accuracy
4–16	100.0%
32	99.8%
64	96.0%
128	55.7%
256	18.8%

3.3 Param-Matched Baselines on Prose (PG-19)

All prose and code models use AdamW with a cosine LR schedule, batch size $B = 2$, sequence length $T = 2048$, and $\sim$6M training tokens unless otherwise noted. All baselines are parameter-matched to the memory model.

We compare three models on PG-19: an 8-layer memory model ($d = 512$, 18.0M params), a 10-layer deep baseline ($d = 512$, 17.2M), and an 8-layer wide baseline ($d = 576$, 17.4M). All trained identically at $B = 2$, $T = 2048$, 1465 steps ($\sim$6M tokens).

Model	Val Loss	PPL
Memory ($d = 512$, 8L)	2.477	11.91
Deep ($d = 512$, 10L)	2.527	12.51
Wide ($d = 576$, 8L)	2.589	13.32

The memory model outperforms both parameter-matched baselines. The memory model is also $\sim$500ms faster per training step than the deep baseline (dense BMMs vs. Mamba's parallel scan), so the gap is larger at iso-FLOPs.

Training dynamics are shown in Figure 1. The gap over the wide baseline grows throughout training (Table 1), indicating all models are substantially undertrained at 6M tokens.

Table 1: Val loss (nats) on PG-19 at training checkpoints.

Tokens	Memory	Deep	Wide
2M	2.806	2.854	2.838
4M	2.561	2.613	2.625
6M	2.477	2.527	2.589

Context length scaling. Evaluating the trained models at longer contexts with no fine-tuning (memory vs. wide baseline):

The gap grows with context length as W accumulates associations. Both models generalise far beyond their 2K training length with no degradation.

W ablation. To confirm that W itself (not just the additional parameters) drives the improvement, we compare two inference passes on the trained code model: normal (W updating each step) vs. frozen (W reset to its pre-step value, so it never accumulates). Over 8 random 8K-token windows:

Freezing W costs 0.895 nats — nearly equal to the full memory-vs-baseline gap of 0.939. The improvement comes from W, not from the extra parameters. The delta is large from position 0 and consistent throughout the sequence, indicating W provides useful associations immediately. At 18M scale, all 8 layers converge to $\gamma \approx 0.991$–0.993, giving effective memory windows of 97–144 steps with little specialization across layers.

At 100M scale the picture changes qualitatively. Layers 2–5 converge to $\gamma \approx 0.997$ (window $\sim$340–355 steps, roughly the length of a function body), while the first layer (window $\sim$80) and last three layers (windows $\sim$50–80) maintain short-range associations. The model has learned to route long-range file-level recall through the middle of the network while delegating local syntax to the early and late layers — confirming the design choice that per-layer γ enables timescale specialization, and showing that specialization only emerges clearly at sufficient scale.

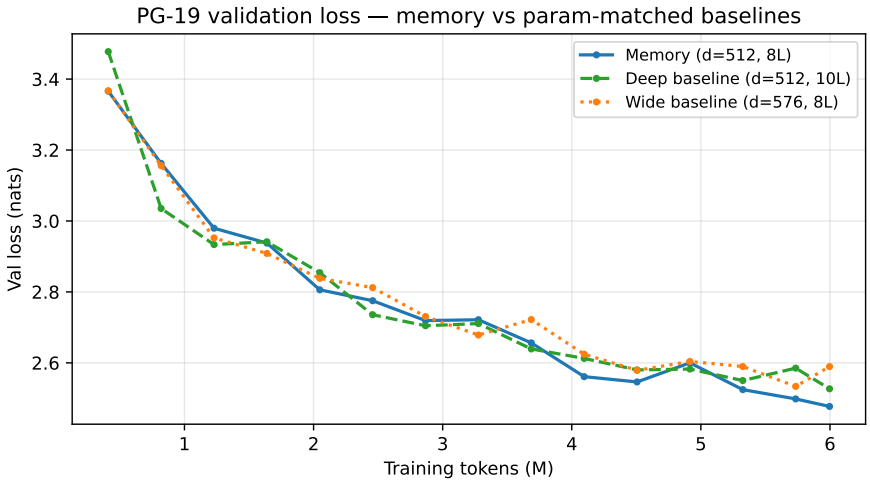

Figure 1: PG-19 val loss over training. The gap over the wide baseline is still widening at 6M tokens.

Eval length	Memory	Wide	Gap
16K (8× train)	2.482	2.500	+0.019
64K (32× train)	2.484	2.512	+0.028

3.4 Context Length Scaling on Prose

We evaluate the trained prose models at longer contexts with no fine-tuning (memory vs. wide baseline, averaged over 8 random windows): [TODO: increase window count — gap is small (+0.019) and per-segment variance is high; 8 windows may be insufficient for significance]

Both models generalise far beyond their 2K training length. The memory advantage is positive and consistent with W accumulating associations over longer sequences. The effect is modest on prose; the gap becomes dramatically larger on code (Section 3.5).

3.5 Code (Headline Result)

We train on Python files from codeparrot/codeparrot-clean (files $\geq$4096 chars), using a 512-vocab BPE tokenizer yielding $\sim$16M training tokens. The memory model is 8-layer (18.0M params) and the baseline is the 10-layer deep model (17.2M params), trained identically.

The gap is $10\times$ larger than on prose. Training curves are shown in Figure 2; the memory model surpasses the baseline's final val loss by step 200 ($\sim$800K tokens, 14% of total training).

Long-context evaluation (18M). We evaluate both 18M models on 4 random 16K-token windows (8× training length) with no fine-tuning, measuring per-segment loss at 1K intervals (averaged over windows):

The per-segment gap grows from $+0.447$ at 0–1K to $+1.067$ at 7–8K as W accumulates file-level associations. Identifiers referenced late in a file were first defined early; by the time the usage site appears, W has stored the (predecessor-of-definition, post-definition state) pair and retrieves it directly. The baseline must instead retain all bindings through its 16-dimensional recurrent state across thousands of intervening tokens.

3.6 Model Size Scaling: 18M $\rightarrow$ 100M

We scale to 100M parameters on the same Python code dataset, training both models identically: B=1, T=2048, 8000 steps (16.4M tokens), constant LR 3×10^{-4}, α=0.05. Val loss is measured on a single batch (20K tokens); readings have high variance but the trend is unambiguous.

	Val Loss	vs. updating
W updating (normal)	2.019	—
W frozen	2.914	+0.895

Eval length	Memory	Wide	Gap
16K (8× train)	2.482	2.500	+0.019

The memory model leads at every evaluation point. Full training dynamics over 16.4M tokens:

The gap measured at final checkpoints (+0.341) understates the true advantage: the memory model's best checkpoint (1.661) beats the baseline's best checkpoint (1.968) by +0.307 nats. The gap at 4M tokens (+0.707) is the largest, reflecting the memory model's early lead before the stronger baseline catches up. Both models are clearly undertrained at 16M tokens.

The most striking result is the cross-scale comparison: the 18M memory model (1.848 nats, 6M tokens) outperforms the fully-trained 100M baseline at *every* point in the 100M baseline's training run, including its best-ever checkpoint (1.968). A model $5.6\times$ smaller, trained on $2.7\times$ fewer tokens, achieves better perplexity because W provides a capability — associative recall over file-length context — that additional Mamba layers and training tokens cannot replicate.

The narrowing gap at 100M vs. 18M (+0.307 vs. +0.939) is partly expected: the 100M baseline is substantially stronger than its 18M counterpart (1.968 best vs. 2.787), leaving less headroom. The dataset constraint — 16M tokens of narrow Python with a 512-token BPE vocabulary — is the primary bottleneck. Longer files and a richer vocabulary provide denser, cleaner key-value pairs for W to exploit.

Long-context evaluation at 100M. We evaluate both 100M models on 8 random 16K-token windows with no fine-tuning, measuring per-segment loss at 512-token intervals (averaged over windows). Full per-segment results and gap are shown in Figure 3; selected rows:

Unlike the 18M results where the gap peaked at 7–8K, the 100M memory model's best segment (1.199) comes at 14.5–15K tokens, and loss is still falling at the end of the 16K window. The larger W (1024×1024) continues accumulating useful associations well past where the 18M capacity saturates. The gap is never below +0.38 nats and the model generalises $8\times$ beyond its 2K training length with no degradation.

Table 2: Final val loss on Python code after 6M training tokens.

Model	Val Loss	BPB	PPL
Memory (d=512, 8L)	1.848	1.33	6.35
Deep (d=512, 10L)	2.787	2.01	16.24
Gap	+0.939	+0.68	

Table 3: Code val loss during training.

Tokens	Memory	Deep	Gap
0.4M	3.207	3.782	+0.575
0.8M	2.739	3.329	+0.590
2.0M	2.195	3.215	+1.021
4.1M	2.024	2.571	+0.547
6.0M	1.848	2.787	+0.939

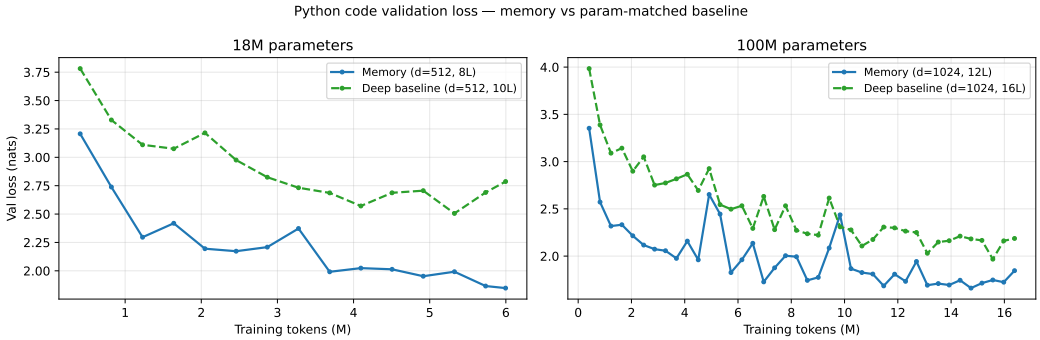

Figure 2: Python code val loss over training. Left: 18M models (6M tokens). Right: 100M models (16.4M tokens, full run). Memory leads at every checkpoint in both scales.

Segment	Memory	Deep	Gap
0K–1K	2.203	2.650	+0.447
2K–3K	2.011	2.632	+0.622
7K–8K	1.631	2.698	+1.067
15K–16K	1.676	2.484	+0.809
Overall	1.912	2.658	+0.746

Model	Params	Final Val	Best Val	PPL (best)
Memory (d=1024, 12L)	106M	1.846	**1.661**	5.27
Deep baseline (d=1024, 16L)	107M	2.187	1.968	7.16
Gap (best vs. best)			+0.307	

Table 4: 100M parameter results on Python code after 16.4M training tokens.

Tokens	Memory	Baseline	Gap
0.4M	3.353	3.984	+0.631
2.0M	2.215	2.898	+0.683
4.1M	2.159	2.866	+0.707
8.2M	1.993	2.273	+0.280
12.3M	1.733	2.264	+0.531
14.7M	**1.661**	2.182	+0.521
15.6M	1.747	**1.968**	+0.221
16.4M	1.846	2.187	+0.341

Table 5: Training dynamics at 100M scale over the full 16.4M token run. Both models are still improving; bold marks each model's best checkpoint.

Segment (K)	0–0.5	0.5–1	1–1.5	1.5–2	2–2.5	2.5–3	3–3.5	3.5–4	4–4.5	4.5–5
Memory	2.036	1.769	1.672	1.648	1.671	1.473	1.808	1.879	1.799	1.854
Deep	2.304	2.157	2.121	2.103	2.120	2.013	2.173	2.251	2.091	2.185
Segment (K)	5–5.5	5.5–6	6–6.5	6.5–7	7–7.5	7.5–8	8–8.5	8.5–9	9–9.5	9.5–10
Memory	1.860	1.750	1.759	1.776	1.618	1.543	1.742	1.746	1.716	1.802
Deep	2.326	2.120	2.158	2.277	2.261	2.045	2.210	2.079	2.029	2.237
Segment (K)	10–10.5	10.5–11	11–11.5	11.5–12	12–12.5	12.5–13	13–13.5	13.5–14	14–14.5	14.5–15
Memory	1.768	1.697	1.780	1.813	1.767	1.731	1.581	1.230	1.405	1.199
Deep	2.197	2.128	2.179	2.162	2.204	2.206	1.941	1.702	1.907	1.676
Segment (K)	15–15.5	15.5–16								Overall
Memory	1.551	1.474								**1.685**
Deep	1.980	1.942								2.109

Table 6: 100M model: per-segment val loss over 16K context (8 windows, 512-token segments). Gap is positive at every segment.

Figure 3: 100M model per-segment val loss (top) and gap (bottom) over 16K context. Vertical dotted line marks training length (2K tokens). The gap is +0.388 at 0.5K, grows to +0.502 at 7.5K, and remains large to the end; memory best is 1.199 at 14.5–15K.

4 Related Work

Fast weights. The closest prior work is Ba et al. [1], who augment an RNN with a fast weight matrix updated by outer products of hidden states: $W_t = \lambda W_{t-1} + \eta\, h_t h_t^\top$, with retrieval via iterative inner-loop steps. Schmidhuber [8] proposed fast-weight memories earlier in a metalearning context. Our W follows the same outer-product-plus-decay principle but makes three changes that are critical in practice: (1) decoupling write and read keys via learned projections — necessary for correct retrieval after Mamba state resets, as shown in Section 3.1; (2) using a fixed injection scale α rather than a learned gate, preventing memory noise from destabilizing early training; and (3) admitting a parallel scan formulation (Section 2) that makes training as efficient as Mamba itself, unlike Ba et al.'s sequential inner-loop retrieval. Trained end-to-end with a standard language modeling objective and no special procedure, this decades-old idea produces large gains on modern language benchmarks.

External and associative memory. Neural Turing Machines [3] and Differentiable Neural Computers [4] maintain an external memory bank with learned content-based read and write heads. Our W is simpler: a single $d \times d$ matrix updated by a fixed rule rather than a large addressable bank with learned attention weights. This trades capacity for constant per-step cost and trivial implementation. Modern Hopfield Networks [7] show that softmax attention computes energy-minimizing retrievals in a Hopfield associative memory. Our W implements an unnormalized causal variant: no softmax normalization, strict causal masking, and separate projections for write and read. These mechanisms are specific to SSMs: in a Transformer, the KV cache already provides lossless associative recall within the context window, leaving no gap for an additional memory matrix.

Memory in SSMs. Test-Time Training [TTT; 10] is an RNN-like model whose hidden state is a weight matrix updated by gradient descent on a self-supervised loss at each step. Titans [2] combine a similar gradient-based neural memory module with sliding-window attention in a hybrid transformer. Both learn the update rule via backpropagation through an inner-loop optimization, incurring additional per-step cost. Our W uses a fixed outer-product rule with no inner loop: $O(d^2)$ per step, purely feedforward, requiring no modification to training. We find this suffices to produce a 0.94-nat gap over a parameter-matched baseline on Python code.

Linear recurrences and the delta rule. RetNet [9], RWKV [6], and Gated Linear Attention [11] reformulate causal attention as a linear recurrence, achieving $O(1)$ state per step. Schlag et al. [13] make the connection explicit: linear attention is equivalent to a fast-weight programmer using Hebbian outer-product writes. DeltaNet [12] replaces the outer product with the error-corrective delta rule: $W_t \leftarrow W_{t-1} + \beta_t(v_t - W_{t-1}k_t)k_t^\top$, actively removing stale associations before writing new ones, which reduces interference more directly than exponential decay. Our update is the simpler pure-Hebbian variant with learned decay, applied additively to Mamba rather than as a replacement for attention. Mamba handles short-range selective compression; W handles long-range associative recall.

5 Conclusion

The contribution is combining existing pieces on a suitable problem. SSMs have a lossy state bottleneck; code is associative recall. A Hebbian matrix W fills the gap directly, at constant memory cost and with no modification to the training objective.

The results are larger than expected. An 18M-parameter memory model reaches a validation loss of 1.848 nats on Python code versus 2.787 for a parameter-matched baseline (+0.939 nats), surpassing the baseline's final performance by step 200 of 1465. Scaling to 100M parameters over 16.4M training tokens, the memory model leads at every checkpoint; best-checkpoint gap is +0.307 nats. The 18M memory model outperforms the 100M baseline at every point in its training run. The gap grows with context length as W accumulates associations at test time. On prose the effect is smaller but consistent (2.477 vs. 2.527 for the deep baseline), suggesting the mechanism is broadly useful and especially powerful when the data is highly associative.

Limitations. W's two projection matrices add roughly 25% to parameter count per layer, comparable to adding extra Mamba layers — but buy a qualitatively different capability those layers cannot

provide. The real constraints are associative capacity (d orthogonal associations per layer) and the single scalar decay per layer, which cannot differentiate retention by content or recency beyond a fixed exponential. We have only evaluated on Mamba; transfer to other SSMs is straightforward but untested.

Future directions. Immediate: longer training sequences (gradient signal for longer-range associations), larger models (capacity scales with d), and harder datasets (multi-document QA, repositories spanning multiple files). A secondary W with slower decay could capture a different timescale of association, with benefit expected to emerge at higher task difficulty. Continual learning is a natural setting: W already implements online associative updates without gradient descent. Further: learnable or input-dependent α, per-layer decay specialization (which our experiments hint at), and whether W can be stored and updated in bf16 or lower precision without capacity loss. Image and video sequences are a natural fit: Vision Mamba processes patches in a fixed scan order and requires multiple passes to capture spatial associations between distant patches — W could short-circuit this by binding patch identities as they are first encountered. Video is even more directly associative, with objects, faces, and scenes appearing, disappearing, and reappearing across frames; W could maintain entity identity across cuts without relying on the recurrent state to carry it.

References

[1] Jimmy Ba, Geoffrey Hinton, Volodymyr Mnih, Joel Z. Leibo, and Catalin Ionescu. Using fast weights to attend to the recent past. In *Advances in Neural Information Processing Systems*, 2016.

[2] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*, 2025.

[3] Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines. *arXiv preprint arXiv:1410.5401*, 2014.

[4] Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Danihelka, Agnieszka Grabska-Barwińska, Sergio Gómez Colmenarejo, Edward Grefenstette, Tiago Ramalho, John Agapiou, et al. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538(7626):471–476, 2016.

[5] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

[6] Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, Kranthi Kiran GV, et al. RWKV: Reinventing RNNs for the transformer era. In *Findings of EMNLP*, 2023.

[7] Hubert Ramsauer, Bernhard Schäfl, Johannes Lehner, Philipp Seidl, Michael Widrich, Thomas Adler, Lukas Gruber, Markus Holzleitner, Milena Pavlovic, Geir Kjetil Stormo, et al. Hopfield networks is all you need. In *International Conference on Learning Representations*, 2021.

[8] Jürgen Schmidhuber. Learning to control fast-weight memories: An alternative to recurrent nets. *Neural Computation*, 4(1):131–139, 1992.

[9] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.

[10] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Aghajanyan, Gregory W. Wornell, Xinlei Chen, and Mike Lewis. Learning to (learn at test time): RNNs with expressive hidden states. *arXiv preprint arXiv:2407.04620*, 2024.

[11] Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. *arXiv preprint arXiv:2312.06635*, 2023.

[12] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. In *Advances in Neural Information Processing Systems*, 2024.

[13] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *International Conference on Machine Learning*, 2021.